

Sistemas Operacionais

Gerência de Memória

Conteúdo

- Mapeamento de memória
- Endereços lógico e físico
- Carga e ligamento dinâmicos
- Overlay e Swapping
- Alocação de memória
- Fragmentação
- Paginação
- Paginação hierárquica
- Segmentação
- Segmentação com Paginação

Memória

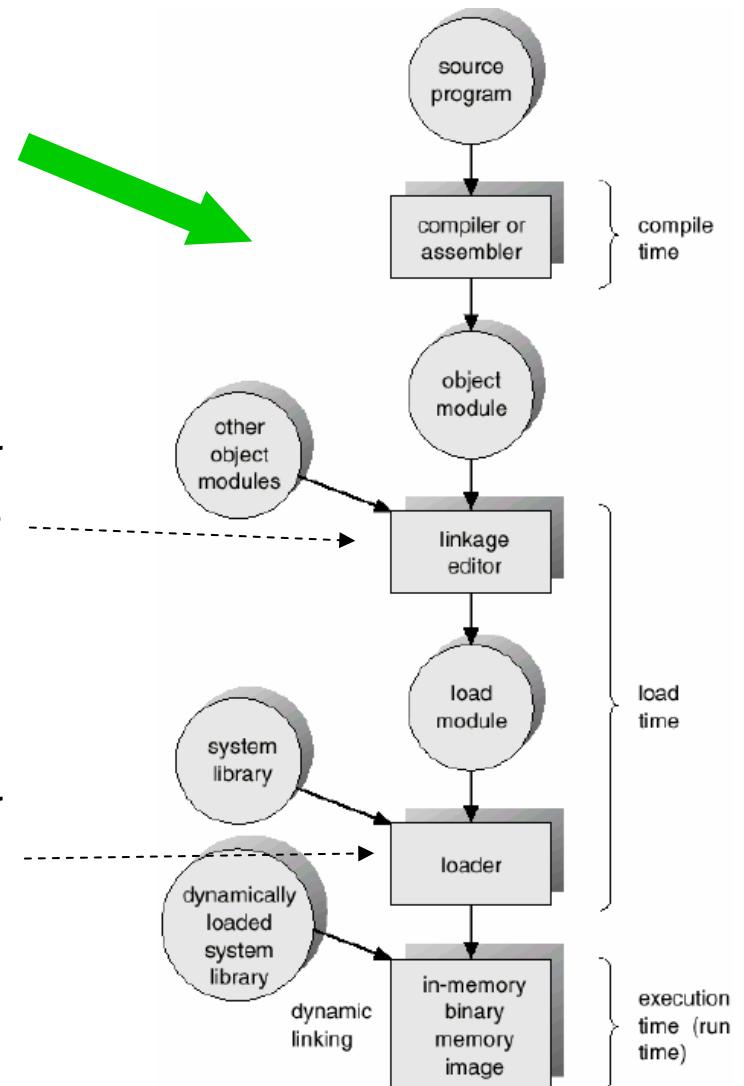
- A CPU busca instruções da memória guiada pelo valor no registrador PC
- Uma instrução pode requerer acessos adicionais à memória para leitura ou escrita
- Portanto, um programa precisa ser carregado em memória para ser executado
- **Fila de entrada** contém grupo de processos em disco (HD) a ser carregado na memória

Mapeamento de endereços de memória

- Normalmente, os programas de usuários podem estar carregados em qualquer parte disponível da memória
- Como tais programas usam endereços simbólicos, um mapeamento p/ end. real é preciso
- Três momentos possíveis para o mapeamento
 - **Na compilação:** se a localização de memória é conhecida a priori, o código absoluto pode ser gerado, mas mudanças no endereço inicial requer recompilação do código (ex.: MS-DOS)
 - **Na carga:** Compilador tem que gerar um código relocável. Mudanças requerem somente recarga do código do usuário
 - **Na execução:** mapeamento é adiado até a execução. Requer suporte de hardware tal como *registradores base e limite*

Mapeamento de endereços de memória (cont.)

- As diversas fases até um programa ser carregado em memória
- O *linker* (ligador) agrupa todos os códigos objetos necessários ao programa final
- O *loader* (carregador) carrega o programa na memória, com as bibliotecas apropriadas do sistema

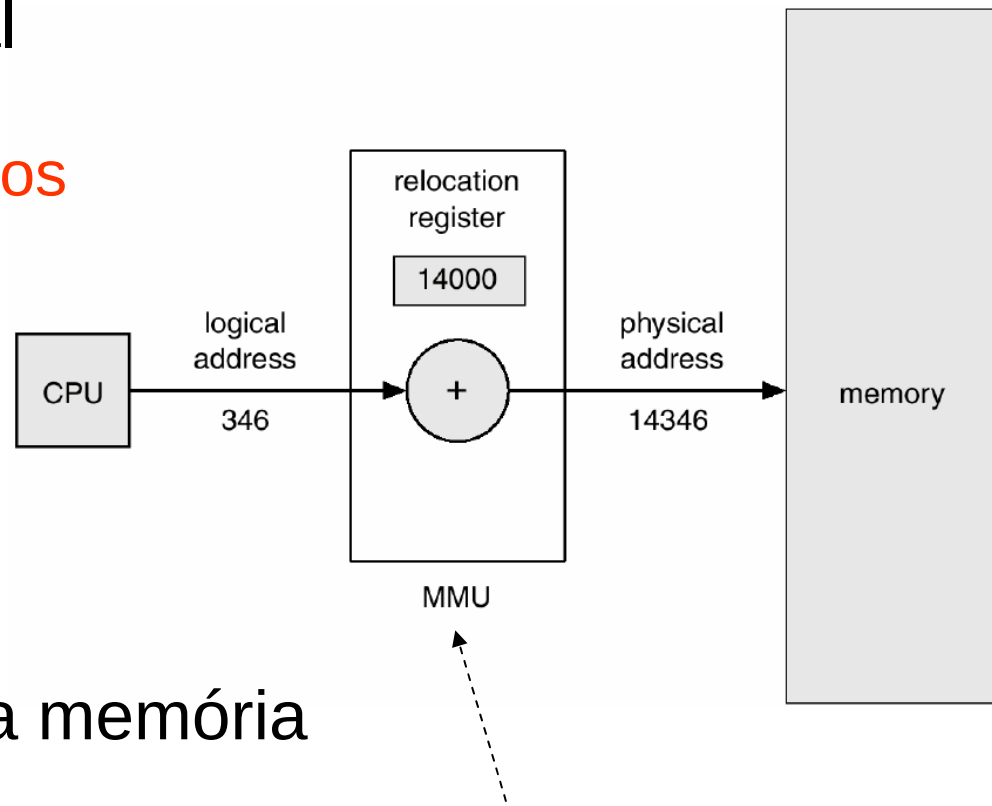


Endereços lógico e físico e a MMU

- **Endereço lógico:**
gerado pela CPU, também
chamado de end. virtual

MMU:
unidade de hardware que
mapeia end. lógico em físico

Ambos endereços são iguais nos
mapeamentos realizados na
compilação e na carga.
Só diferem nos
mapeamentos na execução.



- **Endereço físico:**
endereço real visto pela memória

MMU: Memory Management Unit

Carga dinâmica (dynamic loading)

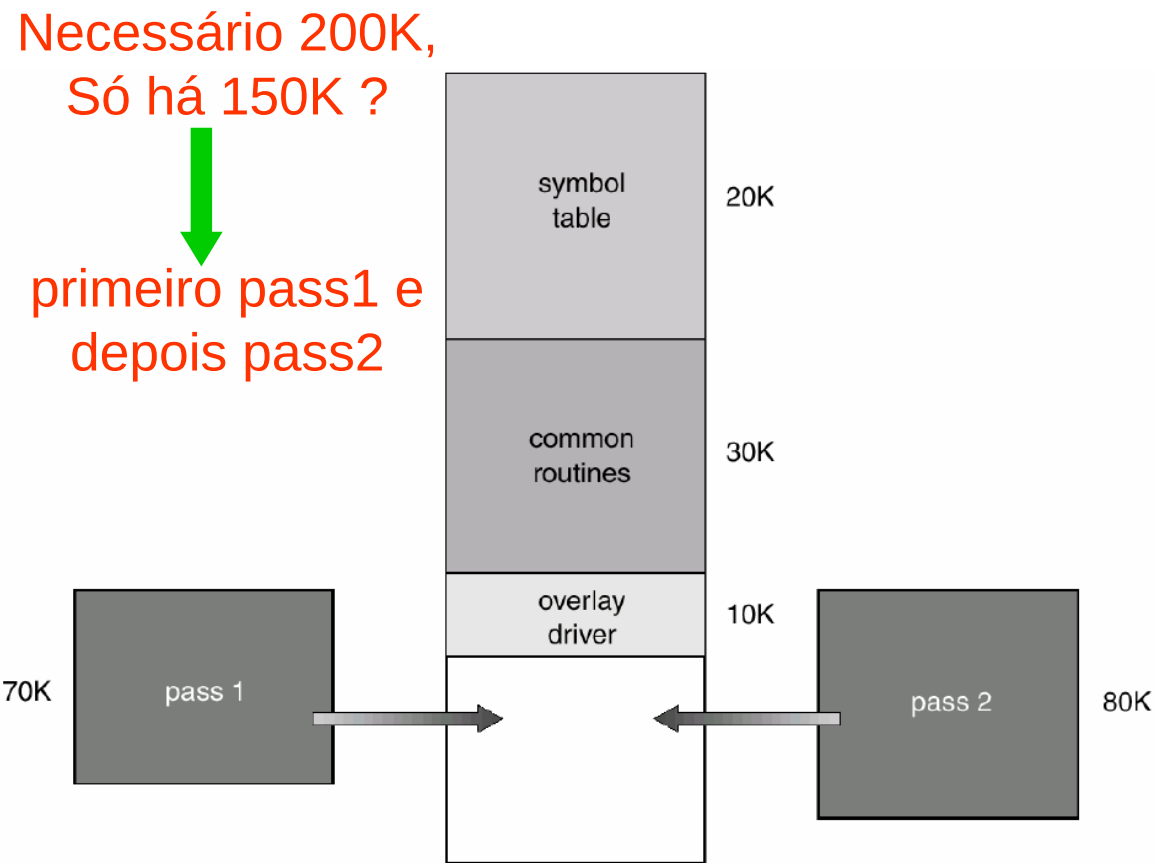
- Rotinas em programas somente são carregadas na memória quando solicitadas
- Provê melhor utilização de memória, pois rotinas que não são utilizadas não necessitam ser carregadas nunca
- Nenhum suporte especial do sistema operacional é necessário.
- Os usuários são responsáveis por desenvolver programas para aproveitar tais recursos

Ligamento dinâmico (dynamic linking)

- Ao invés da carga, o ligamento é adiado até o momento de execução
- Muito útil com bibliotecas do sistema, para que os programas utilizem uma única biblioteca
- Um pequeno pedaço de código, denominado **stub**, é usado para localizar a rotina própria de biblioteca residente em memória
- O **stub** substitui-se com o endereço da rotina e a executa
- O sistema operacional tem que verificar se a rotina está no endereço de memória do processo
- Este método é também chamado: **shared libraries**
 - No windows → **DLLs**

A Técnica Overlay

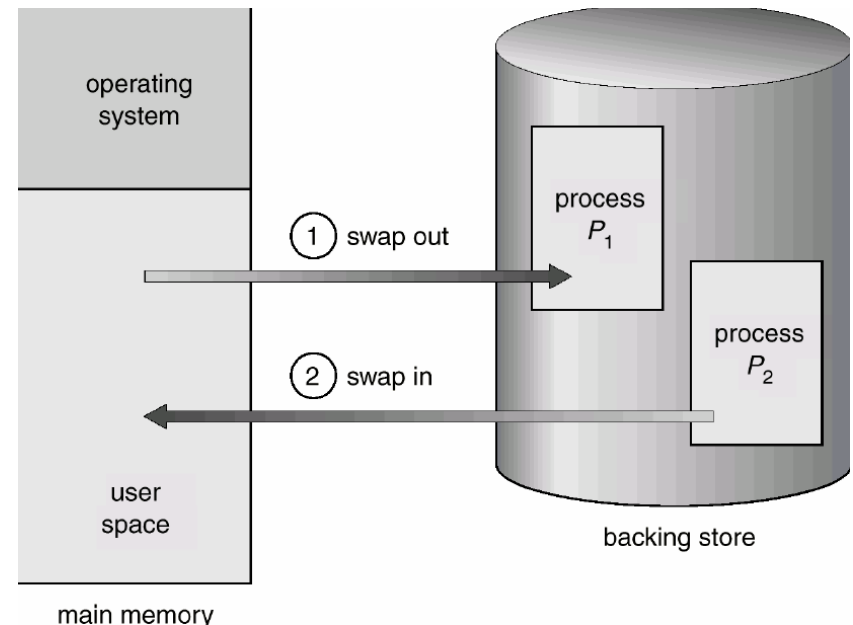
- A idéia é manter em memória somente o que for necessário o tempo todo



- Necessário qdo o processo é maior que a qtidade de memória a ele alocada
- Dispensa suporte especial do SO, mas a programação é complexa
- Outras técnicas são mais usadas !!!

A Técnica Swapping

- Processos podem ser retirados da memória para o disco (swap out) e vice-versa (swap in)
- Prioridade pode ser usada na escolha de qual processo a ser retirado (roll out, roll in)
- Tempo de transferência é significativo
- Tempo do escalonador (quantum) deve ser ajustado adequadamente



Alocação de memória contígua

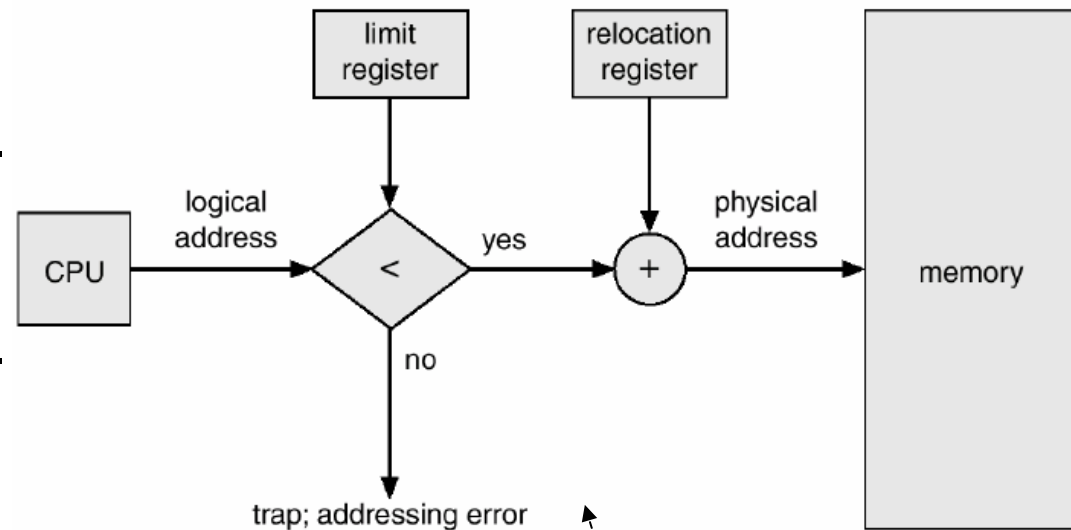
- Cada processo é carregado em uma única partição
- Registradores de relocação e limite asseguram proteção de memória para cada processo

- **Registrador de relocação**

→ possui o menor end. físico

- **Registrador de limite**

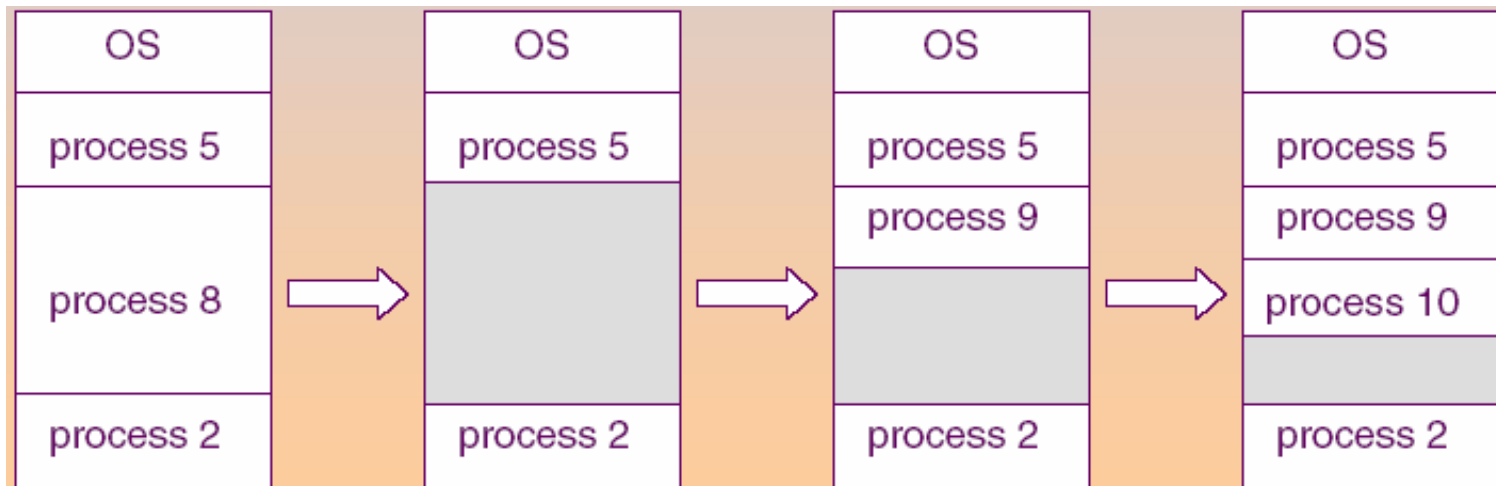
→ define faixa de end. lógicos



operações realizadas
pela MMU

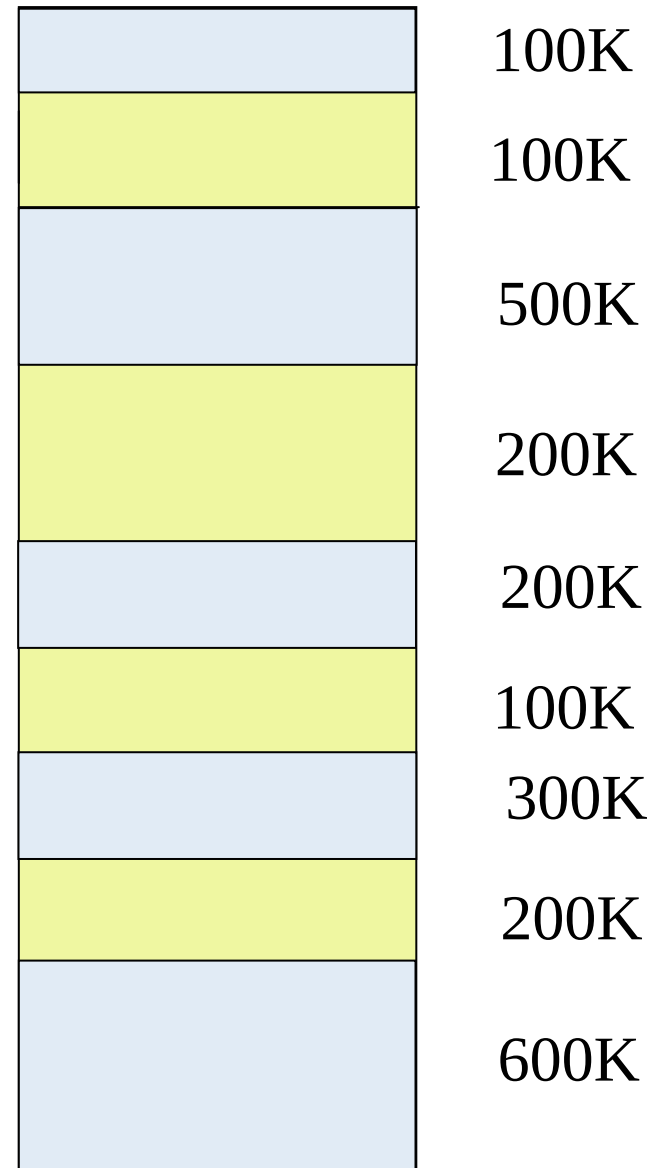
Alocação de memória contígua (cont.)

- Memória torna-se fragmentada
- Novos processos são carregados nas lacunas existentes
- SO mantém info sobre: partições ocupadas e livres
- Algoritmos utilizados para alocação de partições
 - **First-fit** → primeira lacuna suficiente encontrada
 - **Best-fit** → menor lacuna suficiente encontrada
 - **Worst-fit** → maior lacuna encontrada



Exercício

- Dado um esquema de memória com partições de: 100, 500, 200, 300, 600 KB (nesta ordem), pergunta-se:
- Como os algoritmos **first fit**, **best fit** e **worst fit** alocariam processos de tamanho 212, 417, 112 e 426 KB (nesta ordem)?
- Qual algoritmo usa a memória de forma mais eficiente?



Fragmentação

Dois tipos de fragmentação podem ocorrer

- **Fragmentação externa**

- Espaço de memória total é suficiente para alocar um processo, mas não é contíguo

- **Fragmentação interna**

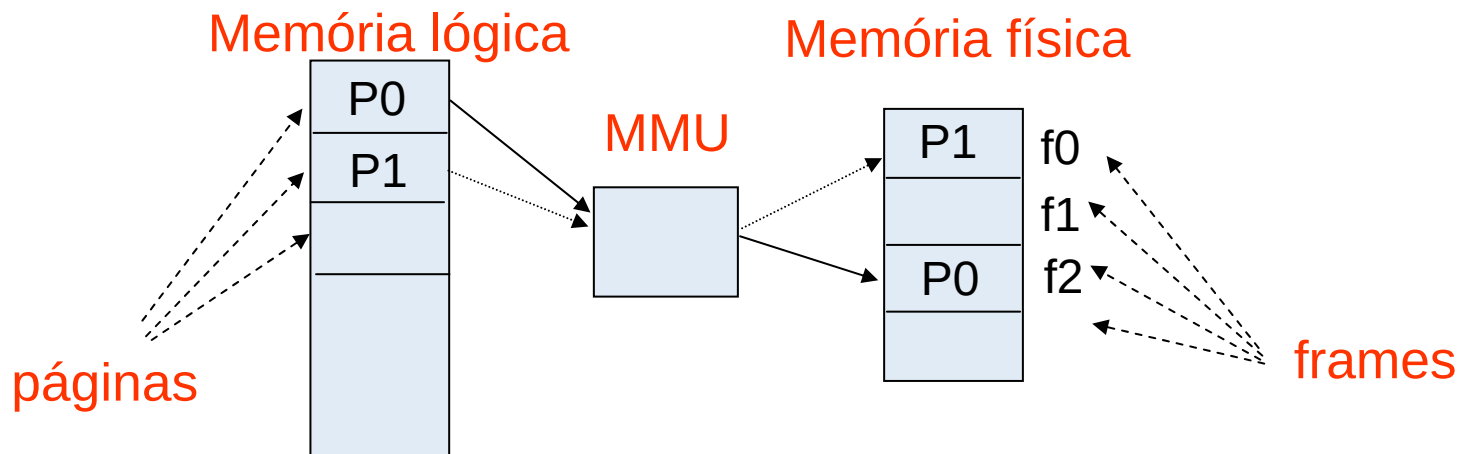
- Partição alocada é um pouco maior do que o processo carregado

- **Fragmentação externa reduzida por compactação**

- Reorganizar toda a memória de modo a juntar todas as lacunas em um único bloco
- Compactação só é possível se a relocação é dinâmica, e feita em tempo de execução

PAGINAÇÃO

- O endereçamento lógico pode ser não contíguo
- **Memória lógica** → dividida em blocos de mesmo tamanho chamados de **páginas**
- **Memória física** → dividida em blocos, de tamanho iguais às páginas, denominados de **frames**
- Um programa cujo tamanho é de n páginas, precisa de n frames livres para ser executado
- Uma **tabela de páginas** é necessário

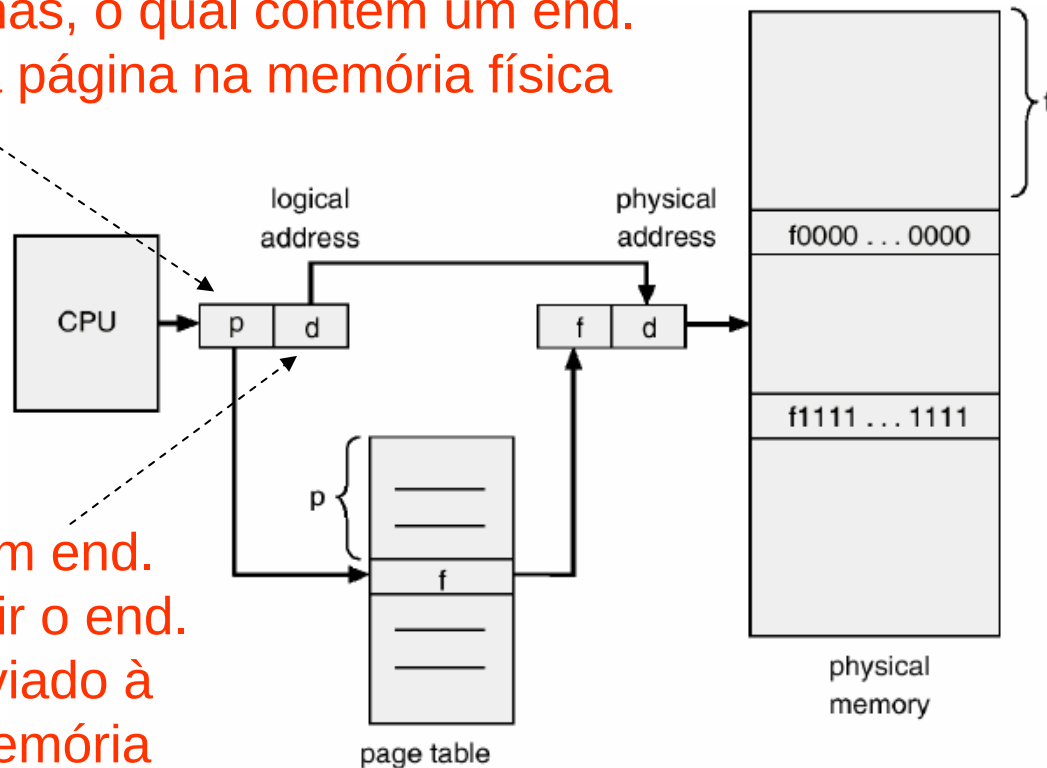


Esquema de translação de endereços

- O endereço gerado pela CPU divide-se em:
 - Número de página (p)
 - Deslocamento dentro da página (d)

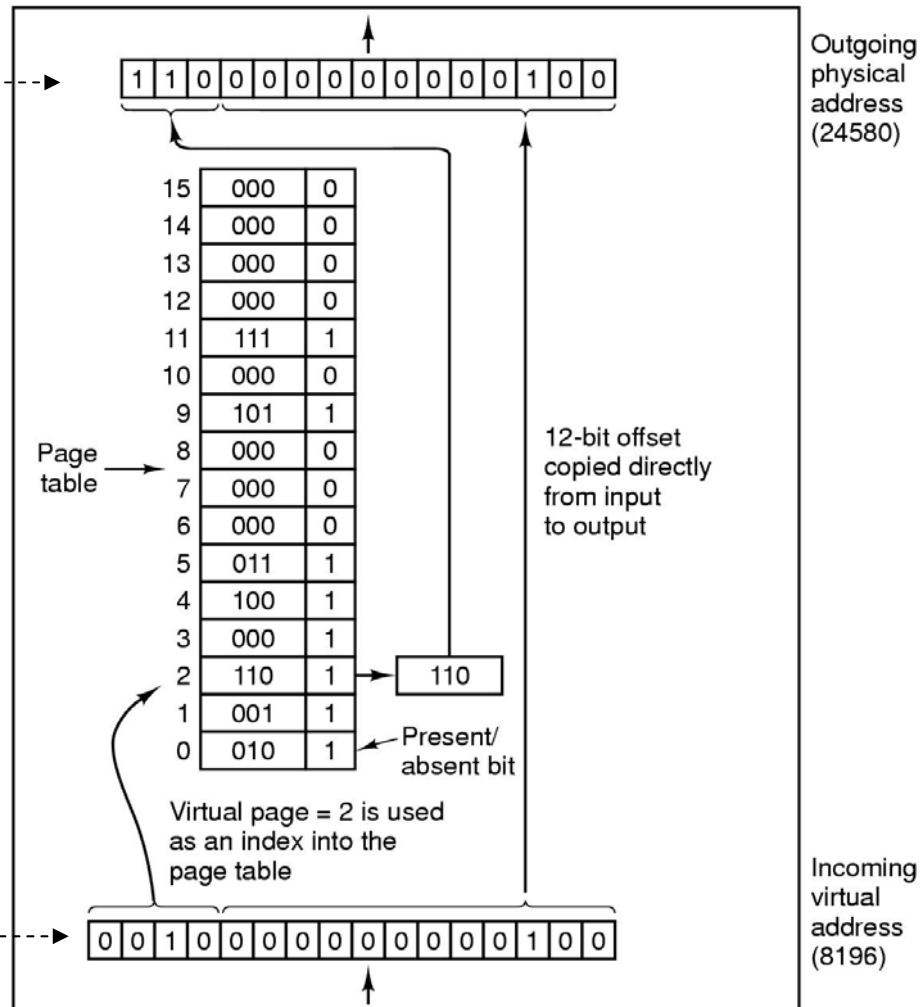
Usado como índice dentro de uma tabela de páginas, o qual contém um end. base de cada página na memória física

Combinado com end. base para definir o end. físico a ser enviado à unidade de memória



Exemplo de translação de endereços

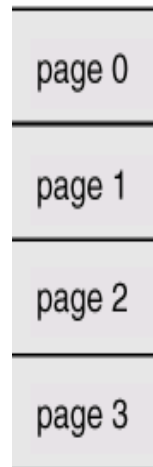
End. Físico
24580



End. Lógico
8196

Exemplo de paginação

Tabela de páginas
guia o mapeamento
de endereços

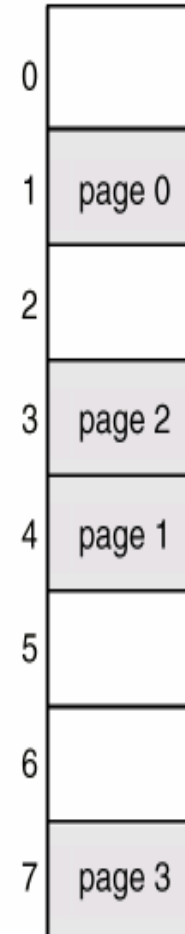


logical
memory

0	1
1	4
2	3
3	7

page table

frame
number



physical
memory

Página e frame têm
o mesmo tamanho

Exemplo de paginação (cont.)

Página 0 ----->

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

Página 3 ----->

0	5
1	6
2	1
3	2

page table

Tamanho da página: 4 bytes

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

Frame 0

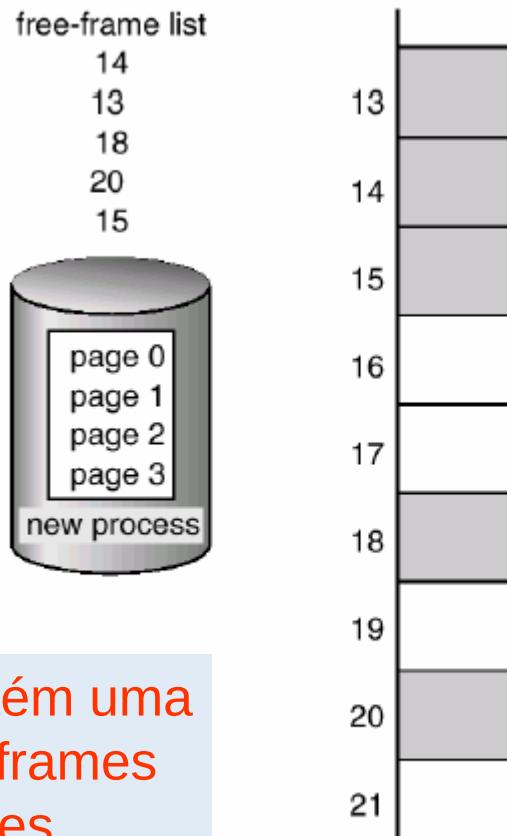
Frame 1

Frame 7

physical memory

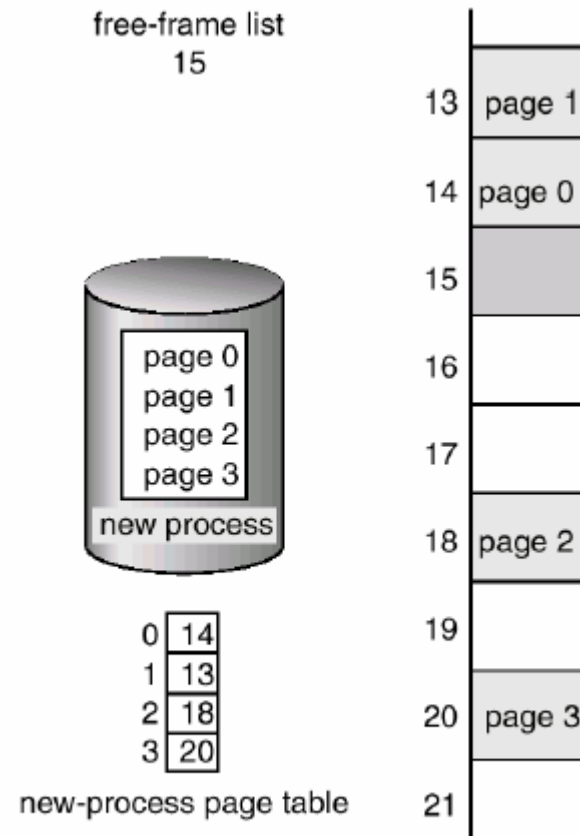
Alocação de frames livres

SO mantém uma
lista de frames
livres



(a)

Antes da alocação



(b)

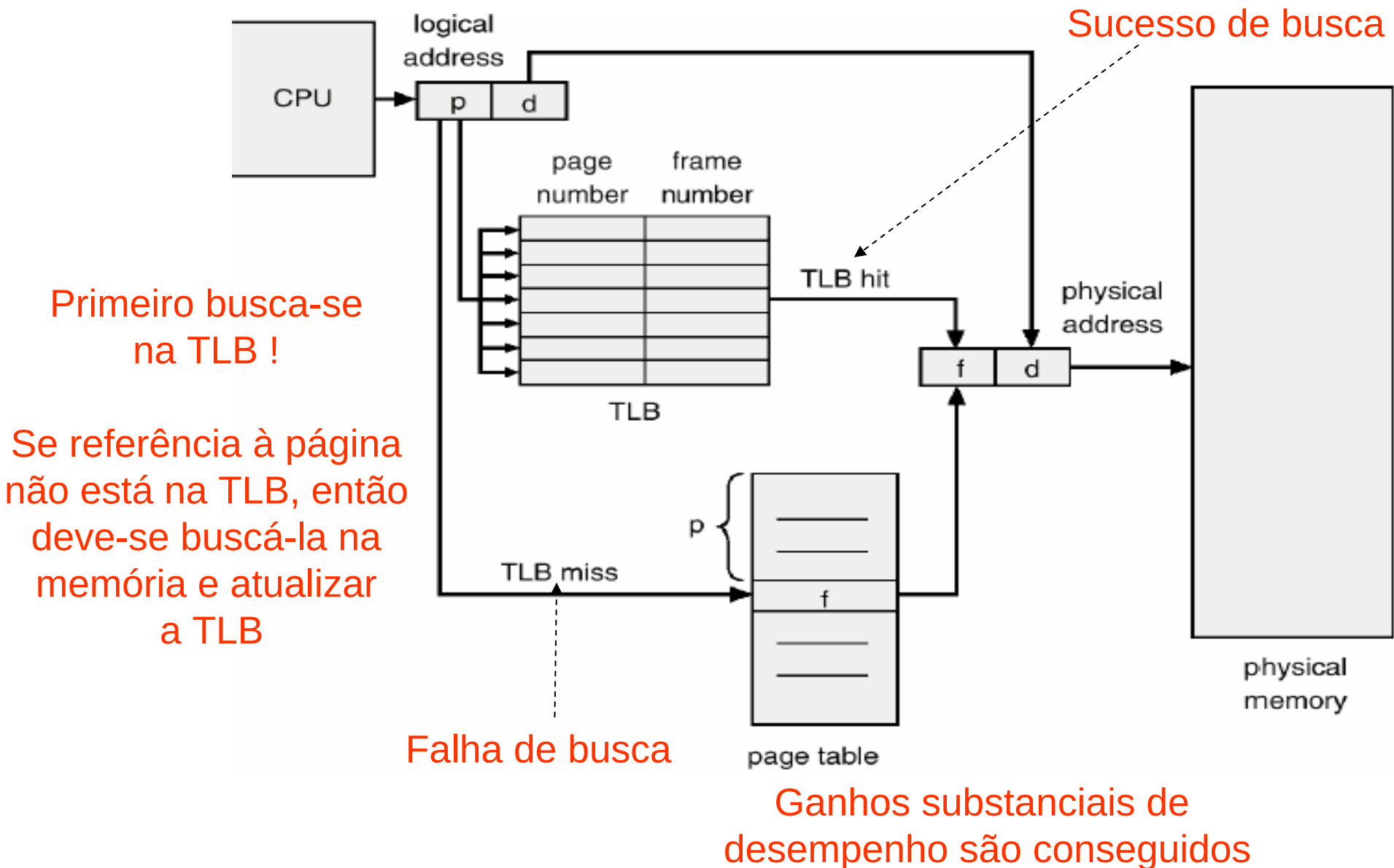
Depois da alocação

Implementação da tabelas de páginas

Duas possibilidades

- Tabela na memória principal
 - Dois acessos à memória, um para ler a tabela e outro para buscar a instrução/dado
- Tabela em memória cache the busca rápida
 - Essa memórias são chamadas **TLBs** (Translation Look-aside Buffers)

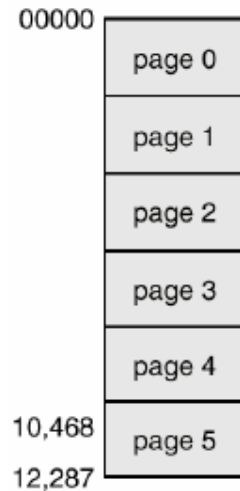
Tabelas de páginas com as TLBs



Proteção de memória

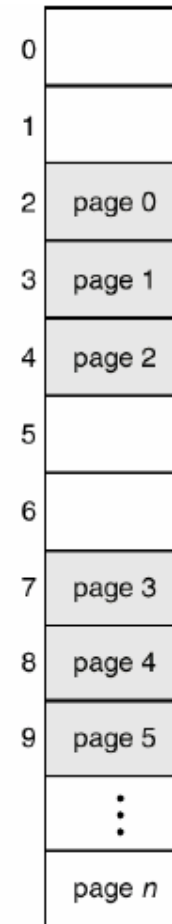
- É conseguida associando-se um bit de proteção, o bit the validade (v), a cada frame.

Raramente um processo usa toda a sua faixa de endereçamento



frame number		valid-invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

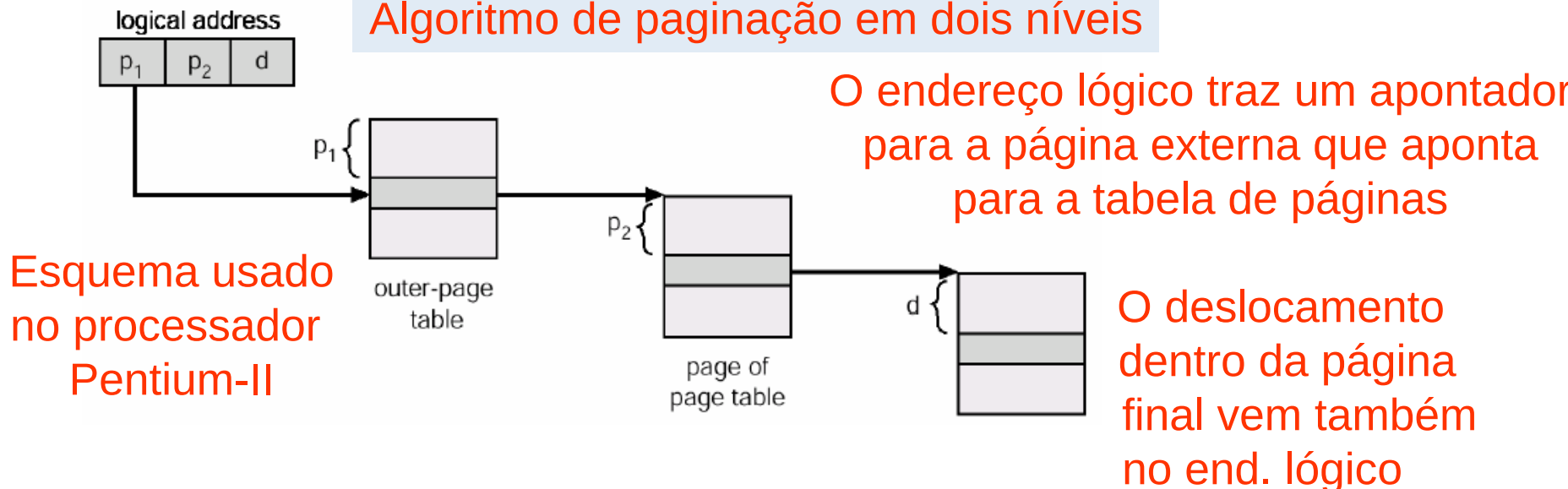


Neste exemplo, o processo não possui valor nenhum nas páginas 6 e 7. Tentativa de acesso a elas, gera um erro !

PAGINAÇÃO HIERÁRQUICA

- Hardwares atuais suportam endereçamentos lógicos bastante extensos (2^{32} a 2^{64} bits)
- **Resultado** → tabelas de páginas extremamente longas
 - Ex.: endereçamento de 32 bits e páginas de 4K
→ tabelas de até 1 milhão de registros
- Solução → dividir a tabela em sub-tabelas
- Sub-tabelas podem ser espalhadas pela memória

Algoritmo de paginação em dois níveis



Paginação em dois níveis: ilustração

- Este esquema é apropriado para até 32 bits de endereçamento
- Para sistemas com mais de 32 bits, a tabela externa torna-se muito longa
- Outras técnicas devem ser usadas
 - Tabelas de páginas Hash
 - Tabelas de páginas invertidas

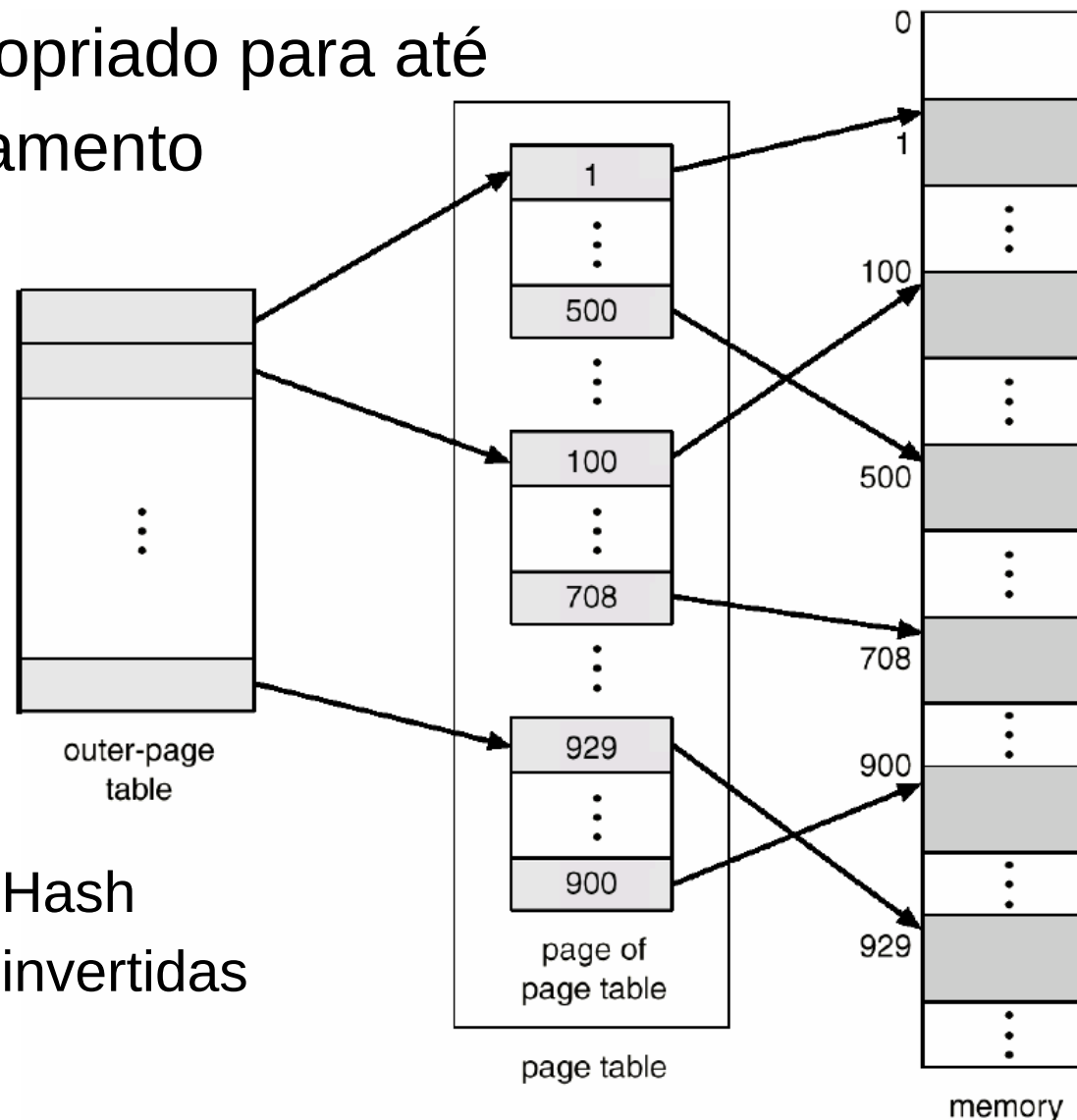
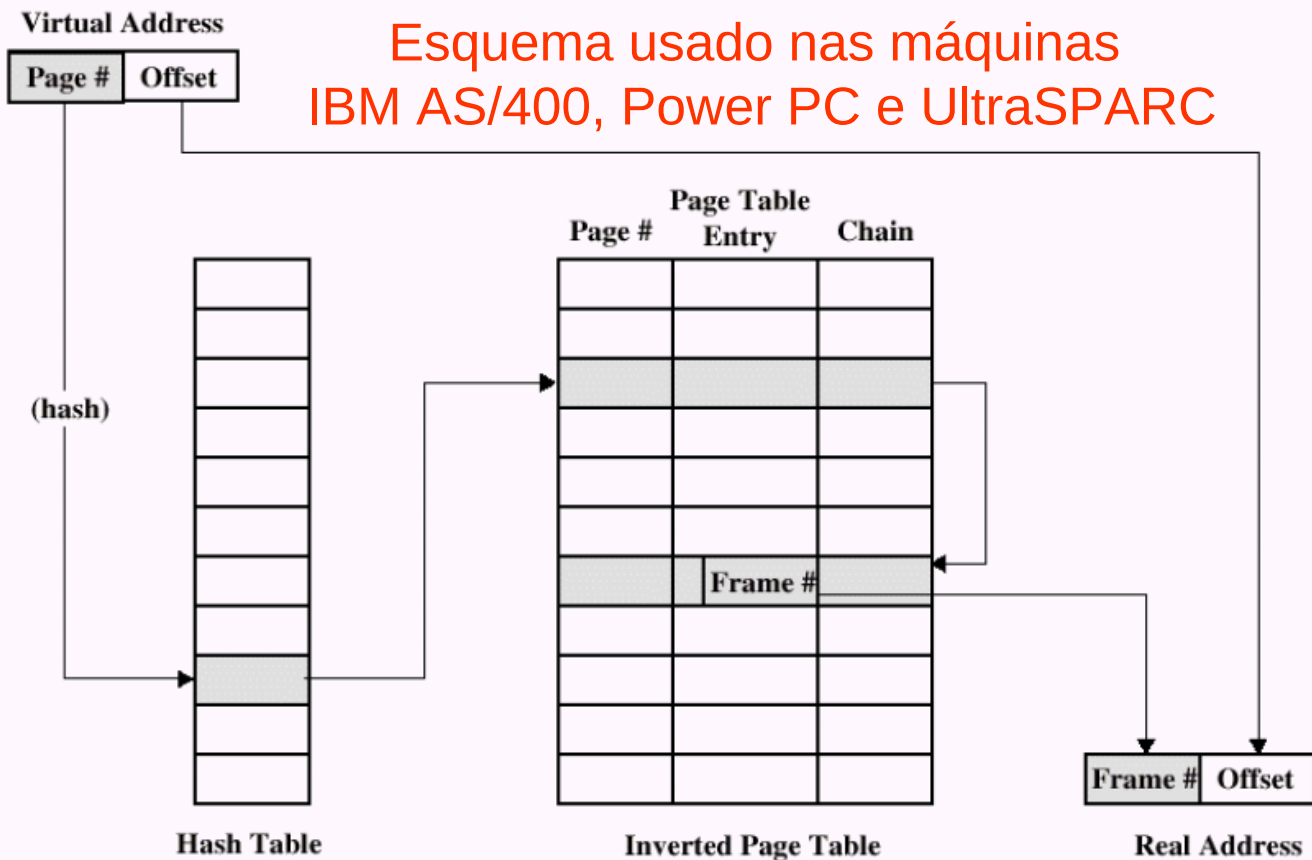


Tabela de páginas Invertida

- Diminui tamanho da tabela de páginas → necessário em sistemas de 64 bits
- Tabela Hash possibilita procura mais rápida

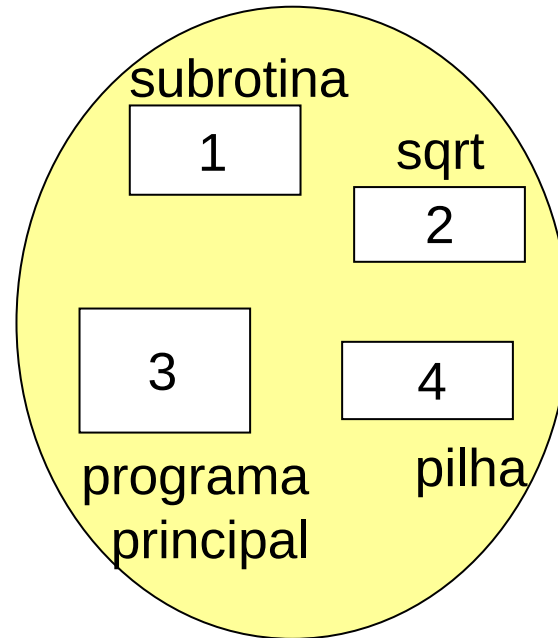


A qtd de registros da tabela de páginas é limitada pela qtd de frames (reais) e não pelo número de páginas virtuais

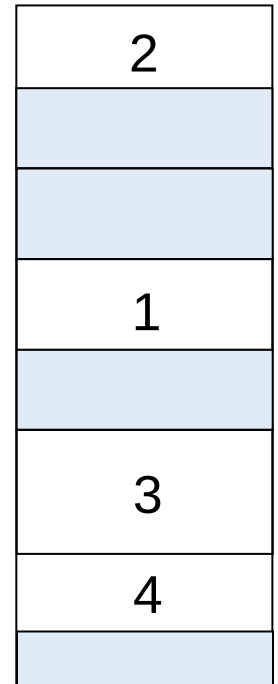
Referências Hash repetidas são manuseados pela técnica Chain

SEGMENTAÇÃO

- Possibilita gerenciar a memória com base na visão do programador → facilita a programação, modularização
- Um programa compõe-se de vários segmentos
 - Programa principal
 - Subrotina
 - Função
 - Pilha
 - Variáveis
 - etc



Programa de usuário

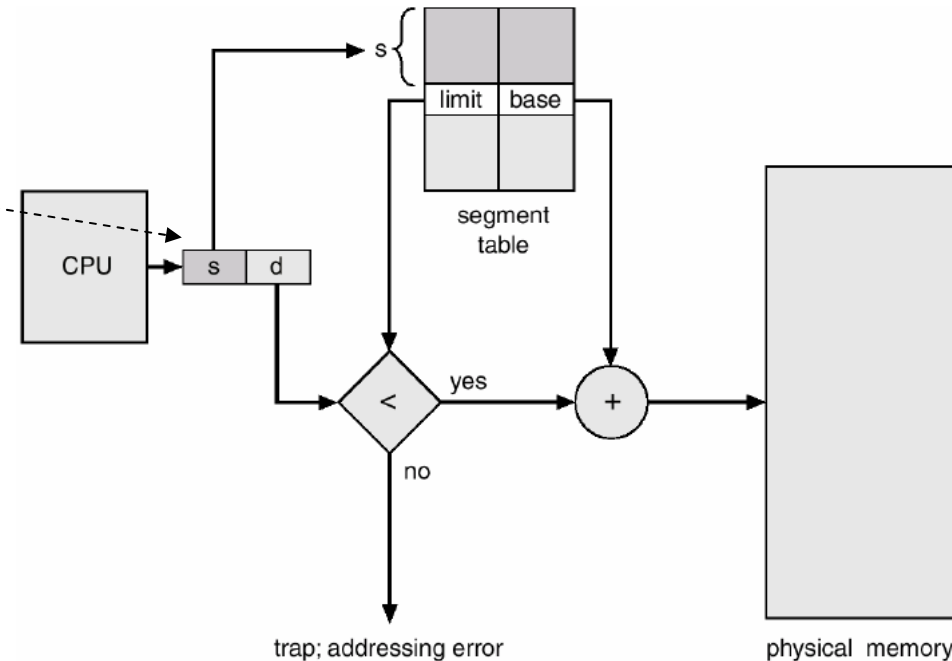


Memória física

Fragmentação externa
está presente !

Implementação da segmentação

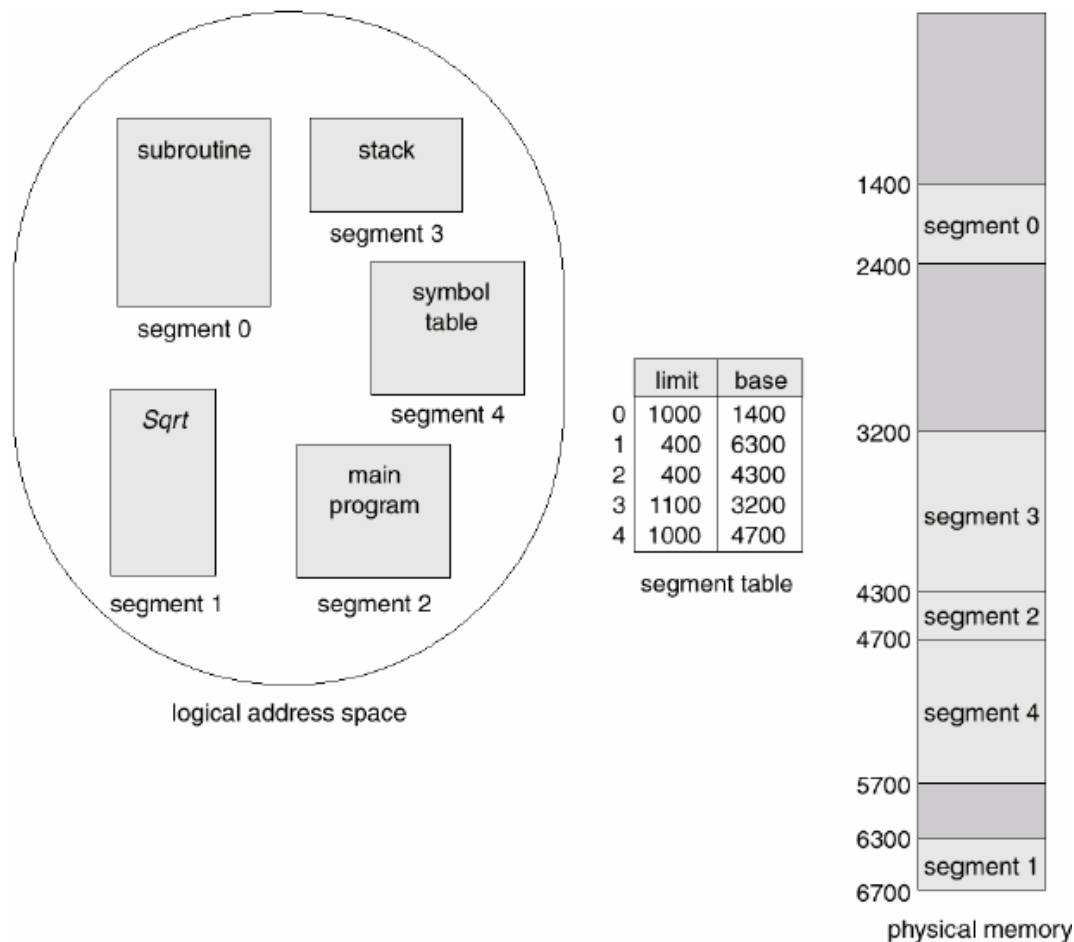
- Como na paginação, uma tabela é utilizada para o mapeamento: end. lógico $\rightarrow$ end. físico
- Cada endereço lógico é formado de:
<Nr. de seg., desl>
- Tabela de segmentos contém em cada registro:
base: end. inicial onde o segmento está na memória física
limit: tamanho do segmento



O mapeamento é
realizado em Hardware !

Exemplo de segmentação

- Cada endereço é mapeado para dentro de seu segmento correspondente conforme a tabela de segs.



Há mais dois registradores para a segmentação

STBR (Segment-Table Base Register)

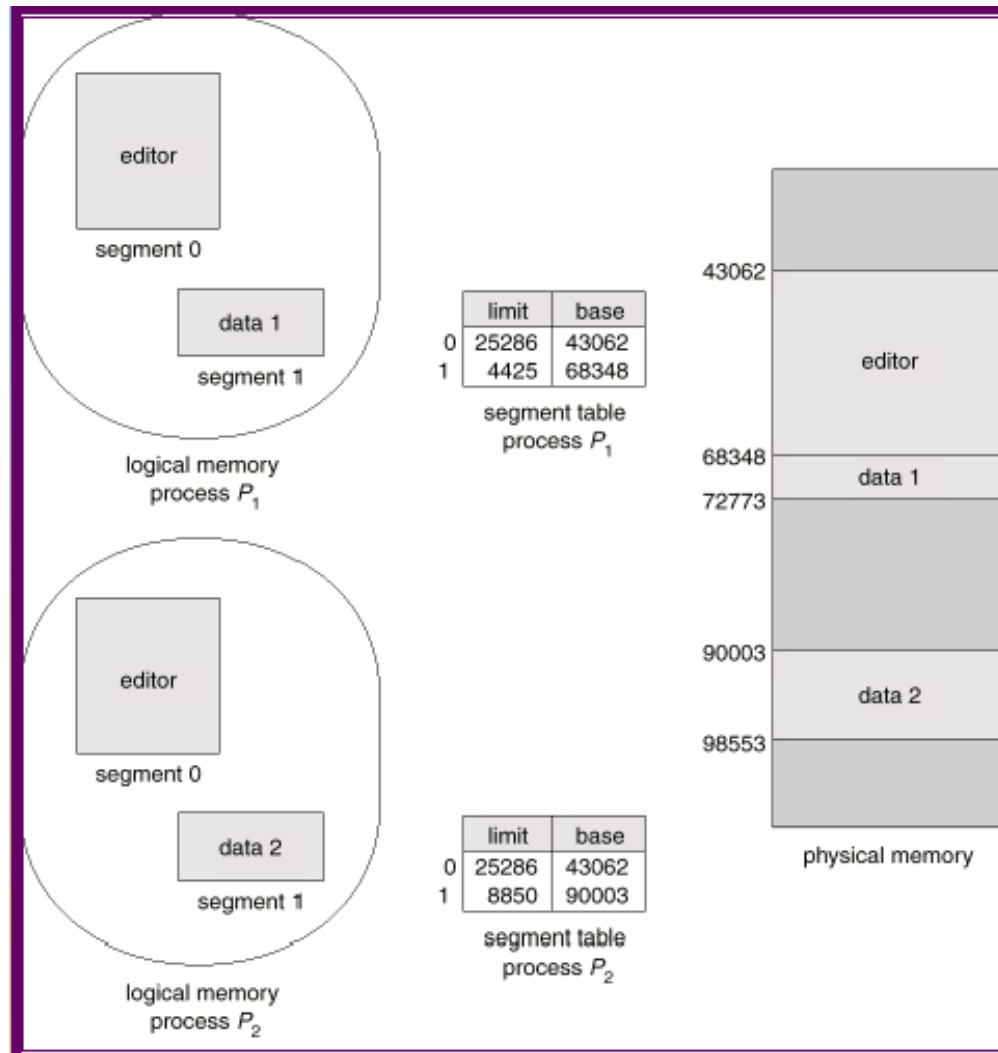
→ aponta para a tabela de segs. na memória

STLR (Segment-Table Length Register)

→ Indica a qtd de segs. usada por um programa

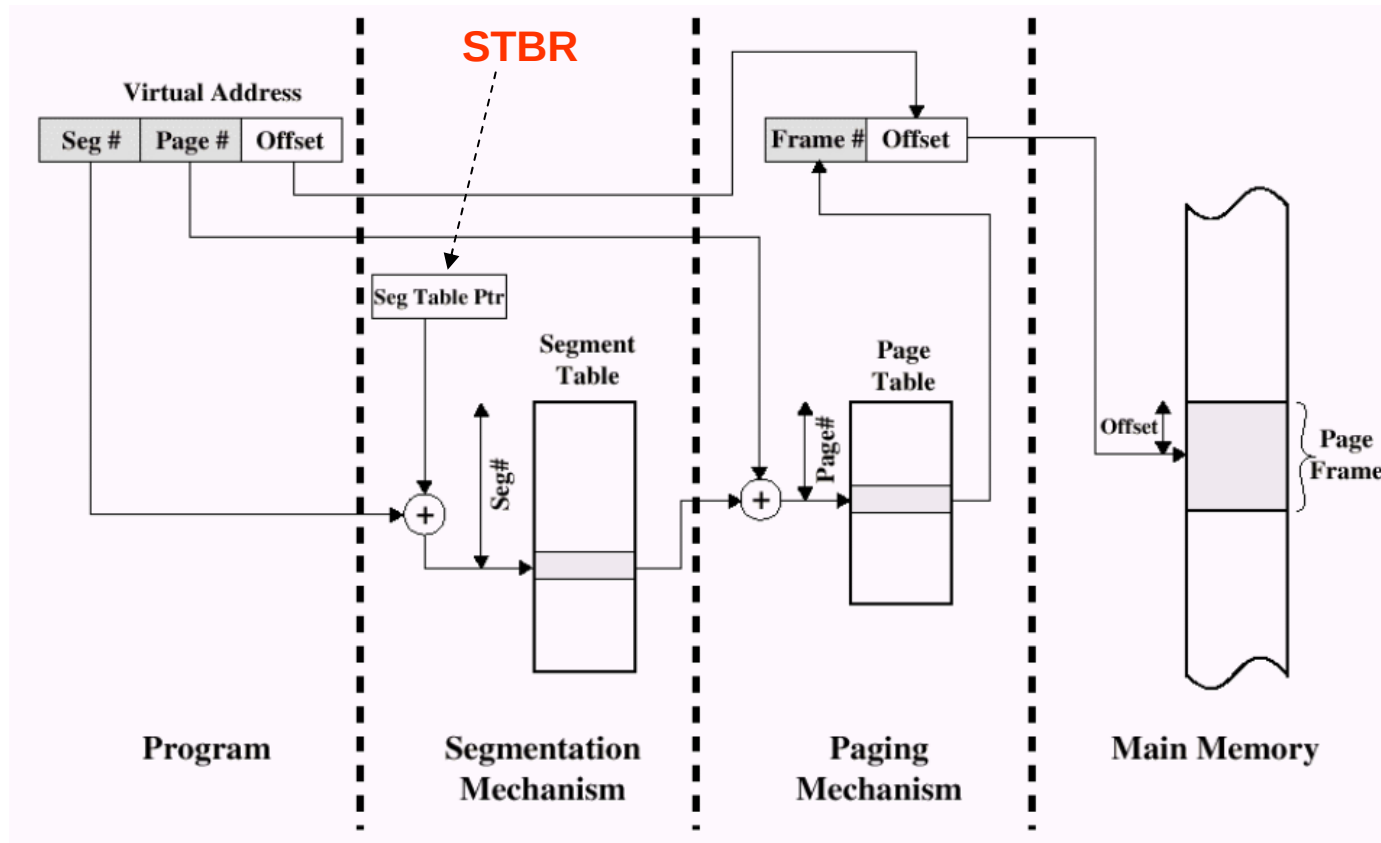
Segmentação com Compartilhamento

- Segmentos podem ser compartilhados
- Segmentos podem ser do tipo: **somente leitura** ou então **somente execução**
- O Hardware detecta erros de acesso
- Ex.: **editor de texto**
somente um segmento para o código, mas vários segmentos para dados de cada usuário
- Compartilham. parcial: subrotina, variáveis, ...



Segmentação com paginação

- Para minimizar o problema da fragmentação externa, os segmentos podem ser paginados



combina-se as vantagens da segmentação e paginação

Programador lida com a segmentação, e o sistema (hardware) gerencia a paginação

Leitura Complementar

- ***Sistemas Operacionais Modernos, 2ª edição***
Andrew Tanenbaum, Editora Pearson, 2003.
- **Operating Systems (4th edition),**
William Stallings, Prentice Hall, 2001
- ***Sistemas Operacionais - Conceitos e Aplicações,***
A. Silberschatz, P. Galvin, e G. Gane, Editora Campus, 2001.
- **Sistemas Operacionais**
R. S. Oliveira, A. Carissimi e S. Toscani, Editora Sagra Luzzatto, 2004